

ISyE6669

Large Scale Optimization Techniques

Column Generation and Constraint Generation

Andy Sun

Spring 2020

In practice, we often encounter large scale optimization problems with many decision variables. In this lecture, we will develop techniques to deal with such situations. First, look at the following problem.

1 Cutting Stock Problem

A paper company produces big rolls of paper, each roll having width W . Customers demand b_i rolls of smaller widths w_i . For example, the big roll has width $W = 100$, and customers want rolls of width $w_1 = 25, w_2 = 35, w_3 = 45$, each for $b_1 = 150, b_2 = 200$, and $b_3 = 300$ rolls. To satisfy demand, the paper company needs to cut big rolls. The goal is to satisfy demand with the minimum number of big rolls to cut. This problem is called the **cutting stock problem**.

Clearly, there are multiple feasible ways to cut a big roll into smaller widths. For example, we can cut 4 copies of width 25 on one big roll, or we can cut 1 smaller roll of width 25 and 2 rolls of width 35 and have a waste of width 5, or we can even just cut 1 smaller roll of width 25 and throw away the rest (just don't tell the manager!). Each such way of cutting a big roll is called a **pattern**.

Let's denote the number of patterns N . A pattern can be expressed as a vector of $[a_1, a_2, a_3]^T$, where each a_i is an integer equal to the number of copies of width w_i cut from a big roll in this pattern. The following table lists some patterns for our example.

Item	Patterns													Demand
w_i	1	2	3	4	5	6	7	8	9	10	11		N	b_i
25	4	3	2	2	2	1	1	1	1	0	0		a_1	150
35	0	0	0	1	0	2	0	1	0	1	0		a_2	200
45	0	0	0	0	1	0	1	0	0	0	1		a_3	300

Let us now formulate an integer optimization problem for the paper company to solve the cutting stock problem. Introduce variable x_j to be the number of large rolls cut in the j -th pattern. So

there are N such variables. We have the following model, called the **Master Problem**:

$$\begin{aligned}
(MP) \quad & \min \quad \sum_{j=1}^N x_j && \text{(total number of big rolls cut)} \\
& \text{s.t.} \quad \sum_{j=1}^N a_{ij} x_j = b_i, \quad \forall i = 1, \dots, m && \text{(Each demand } i \text{ is satisfied exactly)} \\
& && x_j \geq 0, \text{ integer}, \quad \forall j = 1, \dots, N, && \text{(Each } x_j \text{ is nonnegative and integral).}
\end{aligned}$$

Here a_{ij} is the number of width w_i cut in pattern j , and b_i is the demand for width w_i . There are m different types of demand (in our example, $m = 3$).

Or equivalently, the master problem is

$$\begin{aligned}
(MP) \quad & \min \quad \sum_{j=1}^N x_j \\
& \text{s.t.} \quad \sum_{j=1}^N \mathbf{A}_j x_j = \mathbf{b} \\
& && x_j \geq 0, \text{ integer}, \quad \forall j = 1, \dots, N,
\end{aligned}$$

where $\mathbf{A}_j$ is the j -th column of the matrix $\mathbf{A}$, corresponding to the j -th pattern, and $\mathbf{b}$ is the demand vector $\mathbf{b} = [b_1, \dots, b_m]^\top$.

In the following, we ignore the integrality constraints and treat x_j 's as continuous variables. The Master Problem thus becomes a linear program. We call it the LP relaxation of the integer program.

You may be tempted to start solving this problem in Xpress. But, wait, we don't know what N is and we haven't figured out all the patterns yet. You can certainly *try* to figure out all the patterns for our example, which will give you N , and then input them to your Xpress model. However, quickly you will realize that the number of patterns is some combinatorial number in terms of W and the demands, and in general, this makes N a very very big number. Furthermore, writing down all the patterns and inputting them to Xpress is an awefully inefficient way if N is very very big. So what should we do?

The key realization of the so-called **column generation** algorithm is that we can start solving the problem without knowing all the patterns, and only generate patterns that we need on the fly.

In the following, we first describe the column generation algorithm for solving general linear programs. Then, we will specialize it to the cutting stock problem.

2 Column Generation Algorithm

Again, the original problem to solve is the following Master Problem.

$$\begin{aligned}
 (MP) \quad & \min \sum_{j=1}^N c_j x_j \\
 \text{s.t.} \quad & \sum_{j=1}^N \mathbf{A}_j x_j = \mathbf{b} \\
 & x_j \geq 0, \quad \forall j = 1, \dots, N.
 \end{aligned}$$

Column Generation Algorithm

1. Pick a subset of columns $\{\mathbf{A}_j\}$ and variables x_j 's for $j \in I$, where I is a subset of $\{1, 2, \dots, N\}$.
2. Solve the so-called **Restricted Master Problem**.

$$\begin{aligned}
 (RMP) \quad & \min \sum_{j \in I} c_j x_j \\
 \text{s.t.} \quad & \sum_{j \in I} \mathbf{A}_j x_j = \mathbf{b} \\
 & x_j \geq 0, \quad \forall j \in I.
 \end{aligned}$$

Denote the optimal solution of (RMP) as $\hat{\mathbf{x}}_I$.

Three important observations:

- (a) Since (RMP) has all the constraints in the master problem (MP) , the optimal solution $\hat{\mathbf{x}}_I$ of (RMP) satisfies all the demand. Remember $\hat{\mathbf{x}}_I$ only contains components x_j for $j \in I$. We can set all the other components $x_j = 0$ for $j \notin I$. In this way, we get a *feasible* solution to the master problem (MP) . Let's denote this solution as $\hat{\mathbf{x}} = (\hat{\mathbf{x}}_I, \mathbf{0})$.
 - (b) In fact, $\hat{\mathbf{x}}$ is a *basic feasible solution* of (MP) . Why? Think about how the simplex method solves (RMP) . When the simplex method terminates, it finds an optimal solution $\hat{\mathbf{x}}_I$ of (RMP) and the associated basis matrix $\hat{\mathbf{B}}$. This matrix $\hat{\mathbf{B}}$ has the same number of rows as the constraint matrix $\mathbf{A}$ and it is invertible, therefore, $\hat{\mathbf{B}}$ is also a basis matrix for (MP) . More concretely, if we pick columns 9, 10, and 11 to start the (RMP) , i.e. $I = \{9, 10, 11\}$, these three columns will give a basic feasible solution to the Master Problem (MP) .
 - (c) Now we find a BFS $\hat{\mathbf{x}}$ for (MP) with the associated basis matrix $\hat{\mathbf{B}}$. We can compute the reduced costs for the nonbasic variables and see if any nonbasic variable has a negative reduced cost. If yes, we can bring the corresponding column into I , and go back to solve an updated (RMP) . This step of checking reduced cost is called the **pricing subproblem**.
3. **Pricing subproblem:** Compute all the *reduced costs* for $\hat{\mathbf{x}}$ and decide if $\hat{\mathbf{x}}$ is optimal for (MP) . To do this, we can compute the *minimum* reduced cost as

$$w = \min_{j=1, \dots, N} \{c_j - \mathbf{c}_B^T \hat{\mathbf{B}}^{-1} \mathbf{A}_j\}, \quad (1)$$

where $\mathbf{B}$ is the optimal basis obtained from solving the restricted master problem, and we know $\mathbf{B}$ is also a basis for the master problem, but of course, it may not be an optimal basis for the master problem (because the restricted master problem only considers a subset of all columns of the master problem.)

There are two possibilities for w :

- (a) If $w \geq 0$, then all the reduced costs are nonnegative, therefore $\hat{\mathbf{x}}$ is optimal for (MP) . Terminate the column generation algorithm and report $\hat{\mathbf{x}}$ as the optimal solution.
- (b) If $w < 0$, pick the column j with the minimum reduced cost, and add $\mathbf{A}_j$ to the subset of columns, i.e. $I \leftarrow I \cup \{j\}$. Go to Step 2: Solve the new Restricted Master Problem, and repeat the process until we find an optimal solution.

Remarks: Is this column generation algorithm *correct*? By that we mean, first, does it always terminate? second, when it terminates, does it always give the optimal solution to the original problem? The answers to both questions are YES! Here is how we argue about it.

1. First, the column generation algorithm always terminates. Because before termination, every iteration adds a new column to I , and no existing columns in I get removed from I , therefore the subset of columns I always strictly increase every iteration. The worst case is all the N columns are added to I , at that point, the restricted master problem becomes the master problem. So the algorithm will definitely terminates in a finite number of steps.
2. Second, when the column generation algorithm terminates, it will give the optimal solution to MP, because it only terminates when the reduced costs of a BFS are nonnegative, from the simplex method, we know this is an optimal solution to the MP.

3 Apply Column Generation to the Cutting Stock Problem

Now we want to apply the above Column Generation Algorithm to the Cutting Stock Problem. First pick a subset of columns I , and start to solve the restricted master problem. Then, for an optimal solution, we need to compute the minimum reduced cost in Step 3, i.e. the pricing step.

The pricing step is actually quite challenging! Because enumerating through all the reduced costs is again very inefficient, given N is a big big number. The coool idea is NOT to do the enumeration, but formulate another *optimization* problem for computing the minimum reduced cost, using the special structure of the cutting stock problem.

3.1 Generating Column by Solving a Knapsack Problem

First, notice that $c_j = 1$ for all j , because the objective is $\sum_j x_j$. We have

$$w = \min_{j=1,\dots,N} \{c_j - \mathbf{c}_B^T \mathbf{B}^{-1} \mathbf{A}_j\} = \min_{j=1,\dots,N} \{1 - \mathbf{c}_B^T \mathbf{B}^{-1} \mathbf{A}_j\} = 1 - \max_{j=1,\dots,N} \mathbf{c}_B^T \mathbf{B}^{-1} \mathbf{A}_j.$$

Let us denote $\hat{\mathbf{y}}^T = \mathbf{c}_B^T \mathbf{B}$. Now focus on solving $\max_{j=1,\dots,N} \hat{\mathbf{y}}^T \mathbf{A}_j$. We have the following reasoning:

$$\begin{aligned} \max_{j=1,\dots,N} \hat{\mathbf{y}}^T \mathbf{A}_j &= \max \quad \hat{\mathbf{y}}^T \mathbf{A} \\ \text{s.t.} \quad &\mathbf{A} \text{ is a feasible column for the cutting stock problem} \\ &= \max \quad \sum_{i=1}^m \hat{y}_i a_i \end{aligned} \tag{2}$$

$$\text{s.t.} \quad \sum_{i=1}^m w_i a_i \leq W \tag{3}$$

$$a_i \geq 0, \text{ integer.} \tag{4}$$

The first equality follows from the realization that enumerating all the columns is equivalent to finding a feasible column that maximizes the objective. The crucial thing here is that the column becomes a decision variable and a new column is *generated* by an optimization problem. Now we need to characterize what feasible columns are. This is exactly achieved by constraints (3) and (4). Constraint (3) says the total widths used to cut a_i copies of item i for all i should not exceed the total width of a large roll W . Constraint (4) says the number of copies of each item should be a nonnegative integer. These two constraints completely characterize what a feasible column (or pattern) is. The integer program (2) (3) (4) is called a knapsack problem. It can be solved quite efficiently even for large number of a_i 's.

Remark 1. *Notice that the Cutting Stock Problem has some special structures. First, all cost coefficient $c_j = 1$ for all $j = 1, \dots, N$. This makes it possible to focus on the maximization problem over $\mathbf{A}_j$. Second, the set of all columns $\mathbf{A}_j$'s can be described by the knapsack constraints. This makes the pricing subproblem easy to solve. This points to a very important principle in optimization: To solve a large-scale optimization problem we need to exploit the special structure of the problem.*

3.2 Example

Let us solve the cutting stock example given in Section 1. Re-write the cutting stock Master Problem here:

$$\begin{aligned} (MP) \quad \min \quad & \sum_{j=1}^N x_j \\ \text{s.t.} \quad & \sum_{j=1}^N a_{ij} x_j = b_i, \quad \forall i = 1, \dots, m \\ & x_j \geq 0, \text{ integer}, \quad \forall j = 1, \dots, N, \end{aligned}$$

1. Pick an initial set of columns $I = \{9, 10, 11\}$, because this set of columns clearly gives us a basic feasible solution.

- (a) Solve the restricted master problem (RMP):

$$\begin{aligned} \min \quad & x_9 + x_{10} + x_{11} \\ \text{s.t.} \quad & \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} x_9 + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} x_{10} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} x_{11} = \begin{bmatrix} 150 \\ 200 \\ 300 \end{bmatrix} \\ & x_9, x_{10}, x_{11} \geq 0. \end{aligned}$$

The optimal solution is clearly $\hat{x}_9 = 150, \hat{x}_{10} = 200, \hat{x}_{11} = 300$, and it is also a basic feasible solution to the original master problem (MP). The basis matrix $\mathbf{B}$ and $\mathbf{c}_B$ are

$$\mathbf{B} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{c}_B = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad \hat{\mathbf{y}}^T = \mathbf{c}_B^T \mathbf{B}^{-1} = [1, 1, 1]$$

- (b) Compute the minimum reduced cost $\min_{j=1, \dots, N} \{1 - \hat{\mathbf{y}}^T \mathbf{A}_j\}$. Equivalently, solve the knapsack problem:

$$\begin{aligned} \hat{Z} = \max \quad & a_1 + a_2 + a_3 \\ \text{s.t.} \quad & 25a_1 + 35a_2 + 45a_3 \leq 100 \\ & a_1, a_2, a_3 \geq 0, \text{ integers} \end{aligned}$$

The optimal solution is $a_1 = 4, a_2 = 0, a_3 = 0$, which corresponds to $\mathbf{A}_1$. The objective value $Z = 4$. Therefore, the minimum reduced cost is $1 - \hat{Z} = -3 < 0$, which means the current solution $\hat{\mathbf{x}}$ is not optimal. Column $\mathbf{A}_1$ enters the list I .

2. update the subset of columns $I = \{1, 9, 10, 11\}$.

- (a) Solve the restricted master problem (RMP):

$$\begin{aligned} \min \quad & x_1 + x_9 + x_{10} + x_{11} \\ \text{s.t.} \quad & \begin{bmatrix} 4 \\ 0 \\ 0 \end{bmatrix} x_1 + \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} x_9 + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} x_{10} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} x_{11} = \begin{bmatrix} 150 \\ 200 \\ 300 \end{bmatrix} \\ & x_1, x_9, x_{10}, x_{11} \geq 0. \end{aligned}$$

The optimal solution is $\hat{x}_1 = 37.5, \hat{x}_9 = 0, \hat{x}_{10} = 200, \hat{x}_{11} = 300$. The basis matrix $\mathbf{B}$ and $\mathbf{c}_B$ are

$$\mathbf{B} = \begin{bmatrix} 4 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{c}_B = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad \hat{\mathbf{y}}^T = \mathbf{c}_B^T \mathbf{B}^{-1} = [1/4, 1, 1]$$

- (b) Compute the minimum reduced cost $\min_{j=1, \dots, N} \{1 - \hat{\mathbf{y}}^T \mathbf{A}_j\}$. Equivalently, solve the knapsack problem:

$$\begin{aligned} \hat{Z} = \max \quad & \frac{1}{4}a_1 + a_2 + a_3 \\ \text{s.t.} \quad & 25a_1 + 35a_2 + 45a_3 \leq 100 \\ & a_1, a_2, a_3 \geq 0, \text{ integers} \end{aligned}$$

The optimal solution is $a_1 = 1, a_2 = 2, a_3 = 0$, which corresponds to $\mathbf{A}_6$. The objective value $Z = 9/4$. Therefore, the minimum reduced cost is $1 - \hat{Z} = -5/4 < 0$, which means the current solution $\hat{\mathbf{x}}$ is not optimal. Column $\mathbf{A}_6$ enters the list I .

3. update the subset of columns $I = \{1, 6, 9, 10, 11\}$.

(a) Solve the restricted master problem (RMP):

$$\begin{aligned} \min \quad & x_1 + x_6 + x_9 + x_{10} + x_{11} \\ \text{s.t.} \quad & \begin{bmatrix} 4 \\ 0 \\ 0 \end{bmatrix} x_1 + \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} x_6 + \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} x_9 + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} x_{10} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} x_{11} = \begin{bmatrix} 150 \\ 200 \\ 300 \end{bmatrix} \\ & x_1, x_6, x_9, x_{10}, x_{11} \geq 0. \end{aligned}$$

The optimal solution is $\hat{x}_1 = 12.5, \hat{x}_6 = 100, \hat{x}_9 = 0, \hat{x}_{10} = 0, \hat{x}_{11} = 300$. The basis matrix $\mathbf{B}$ and $\mathbf{c}_B$ are

$$\mathbf{B} = \begin{bmatrix} 4 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{c}_B = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad \hat{\mathbf{y}}^T = \mathbf{c}_B^T \mathbf{B}^{-1} = [1/4, 3/8, 1]$$

(b) Compute the minimum reduced cost $\min_{j=1, \dots, N} \{1 - \hat{\mathbf{y}}^T \mathbf{A}_j\}$. Equivalently, solve the knapsack problem:

$$\begin{aligned} \hat{Z} = \max \quad & \frac{1}{4}a_1 + \frac{3}{8}a_2 + a_3 \\ \text{s.t.} \quad & 25a_1 + 35a_2 + 45a_3 \leq 100 \\ & a_1, a_2, a_3 \geq 0, \text{ integers} \end{aligned}$$

The optimal solution is $a_1 = 0, a_2 = 0, a_3 = 2$, which corresponds to $\mathbf{A}_{14}$. The objective value $Z = 2$. Therefore, the minimum reduced cost is $1 - \hat{Z} = -1 < 0$, which means the current solution $\hat{\mathbf{x}}$ is not optimal. Column $\mathbf{A}_{14}$ enters the list I .

4. update the subset of columns $I = \{1, 6, 9, 10, 11, 14\}$.

(a) Solve the restricted master problem (RMP):

$$\begin{aligned} \min \quad & x_1 + x_6 + x_9 + x_{10} + x_{11} + x_{14} \\ \text{s.t.} \quad & \begin{bmatrix} 4 \\ 0 \\ 0 \end{bmatrix} x_1 + \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} x_6 + \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} x_9 + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} x_{10} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} x_{11} + \begin{bmatrix} 0 \\ 0 \\ 2 \end{bmatrix} x_{14} = \begin{bmatrix} 150 \\ 200 \\ 300 \end{bmatrix} \\ & x_1, x_6, x_9, x_{10}, x_{11}, x_{14} \geq 0. \end{aligned}$$

The optimal solution is $\hat{x}_1 = 12.5, \hat{x}_6 = 100, \hat{x}_9 = 0, \hat{x}_{10} = 0, \hat{x}_{11} = 0, \hat{x}_{14} = 150$. The basis matrix $\mathbf{B}$ and $\mathbf{c}_B$ are

$$\mathbf{B} = \begin{bmatrix} 4 & 0 & 0 \\ 1 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}, \quad \mathbf{c}_B = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad \hat{\mathbf{y}}^T = \mathbf{c}_B^T \mathbf{B}^{-1} = [1/4, 3/8, 1/2]$$

- (b) Compute the minimum reduced cost $\min_{j=1,\dots,N}\{1 - \hat{\mathbf{y}}^T \mathbf{A}_j\}$. Equivalently, solve the knapsack problem:

$$\begin{aligned} \hat{Z} = \max \quad & \frac{1}{4}a_1 + \frac{3}{8}a_2 + \frac{1}{2}a_3 \\ \text{s.t.} \quad & 25a_1 + 35a_2 + 45a_3 \leq 100 \\ & a_1, a_2, a_3 \geq 0, \text{ integers} \end{aligned}$$

The optimal solution is $a_1 = 2, a_2 = 0, a_3 = 1$. The objective value $Z = 1$. Therefore, the minimum reduced cost is $1 - \hat{Z} = 0$, which means the current solution $\hat{\mathbf{x}}$ is optimal!

5. Since we seek for an integer solution as the number of rolls, we round up the LP solution to the nearest integers. $x_1 = 13, x_6 = 100, x_{14} = 150$, and other variables are zero.

4 The Dual Perspective: Constraint Generation

We developed LP duality in the previous lectures. Now we use to gain a better understanding of the above column generation algorithm. In particular, if we look at the problem from the dual perspective, we see what column generation applied on the primal problem does to the dual problem.

$\begin{aligned} (MP) \quad Z = \min \quad & \sum_{j=1}^N c_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^N a_{ij} x_j = b_i, \quad \forall i = 1, \dots, m \\ & \text{(Equivalently: } \sum_{j=1}^N \mathbf{A}_j x_j = \mathbf{b}) \\ & x_j \geq 0, \quad \forall j = 1, \dots, N, \end{aligned}$	$\begin{aligned} (RMP) Z_R = \min \quad & \sum_{j \in I} c_j x_j \\ \text{s.t.} \quad & \sum_{j \in I} a_{ij} x_j = b_i, \quad \forall i = 1, \dots, m \\ & \text{(Equivalently: } \sum_{j \in I} \mathbf{A}_j x_j = \mathbf{b}) \\ & x_j \geq 0, \quad \forall j \in I. \end{aligned}$
$\downarrow \text{Dual}$	$\downarrow \text{Dual}$
$\begin{aligned} (DMP) \quad Z_D = \max \quad & \sum_{i=1}^m b_i y_i \\ \text{s.t.} \quad & \sum_{i=1}^m a_{ij} y_i \leq c_j, \quad \forall j = 1, \dots, N \\ & \text{(Equivalently: } \mathbf{y}^T \mathbf{A}_j \leq c_j, \quad \forall j = 1, \dots, N) \end{aligned}$	$\begin{aligned} (DRMP) Z_{DR} = \max \quad & \sum_{i=1}^m b_i y_i \\ \text{s.t.} \quad & \sum_{i=1}^m a_{ij} y_i \leq c_j, \quad \forall j \in I. \\ & \text{(Equivalently: } \mathbf{y}^T \mathbf{A}_j \leq c_j, \quad \forall j \in I.) \end{aligned}$

Here we summarize a few important observations on the primal and dual perspectives.

1. The Dual Master Problem (*DMP*) is exactly the dual of the primal Master Problem (*MP*), and the Dual Restricted Master Problem (*DRMP*) is exactly the dual of the primal Restricted Master Problem (*RMP*). Therefore, if they have finite optimum, by the strong duality theorem, we have $Z = Z_D$, and $Z_R = Z_{DR}$.

2. We can see that the (DMP) has many constraints since N is a large number, as the primal master problem has many variables. And, (DRMP) has a subset I of all the N constraints.
3. As we start with a subset of columns for the primal restricted master problem, correspondingly we start with a subset of constraints for the dual restricted master problem. As we generate columns with negative reduced costs for the primal problem, we generate constraints that are being violated by the current dual solution. This procedure is called *constraint generation*. If a problem has many constraints and not many variables, we can use constraint generation, rather than column generation.

Let us be more specific here.

- (a) For the primal problem, we know that an optimal solution $\hat{\mathbf{x}}$ of (RMP) is also feasible for (MP), because it satisfies all the constraints in (MP) (the un-selected variables are set to zero, i.e. $x_j = 0$ for $j \notin I$.) To check if $\hat{\mathbf{x}}$ is optimal for (MP), we look at its minimum reduced cost

$$w = \min_{j=1,\dots,N} \{c_j - \mathbf{c}_B^T \mathbf{B}^{-1} \mathbf{A}_j\} \quad (5)$$

If $w \geq 0$, then $\hat{\mathbf{x}}$ is optimal for (MP), otherwise, add the column with the lowest reduced cost to the (RMP), and continue the column generation algorithm. Note the order of reasoning here: to show/check the optimality of $\hat{\mathbf{x}}$ to (MP), we need to first check its feasibility to (MP), then check its optimality. In other words, optimality is always built on feasibility.

- (b) For the dual problem, given an optimal solution $\hat{\mathbf{y}}$ of (DRMP), is $\hat{\mathbf{y}}$ always feasible for (DMP)? NO. Because it only satisfies a subset I of the constraints in (DMP), and may not satisfy other constraints not in I . So, we need to check which constraint is violated by $\hat{\mathbf{y}}$, and add that constraint to (DRMP). How to do this? We can check the following:

$$u = \min_{j=1,\dots,N} \{c_j - \hat{\mathbf{y}}^T \mathbf{A}_j\}. \quad (6)$$

If $u \geq 0$, then we know $\hat{\mathbf{y}}^T \mathbf{A}_j \leq c_j$ for all $j = 1, \dots, N$, which means $\hat{\mathbf{y}}$ is feasible for (DMP). Otherwise, if $u < 0$, then $\hat{\mathbf{y}}$ violates some constraint. Add this constraint to (DRMP). Note that (5) and (6) look very similar. We explore this connection in the next section.

5 Column Generation Using Dual Information

Now we understand the dual perspective of the column generation procedure. We want to use the dual information for the update in primal column generation. In particular, if we are given an optimal $\hat{\mathbf{x}}$ to (RMP), and a corresponding dual optimal $\hat{\mathbf{y}}$ to (DRMP). How to check if $\hat{\mathbf{x}}$ is optimal for (MP)? Of course, we can form the basis matrix and use (5). But since we already have the dual information, we do not need to form the basis matrix, instead we can use (6).

Replace Step 3 of the Column Generation Algorithm (page 3) with the following procedure:

3' Compute the following minimum using the dual solution $\hat{\mathbf{y}}$:

$$u = \min_{j=1,\dots,N} \left\{ c_j - \hat{\mathbf{y}}^T \mathbf{A}_j \right\}$$

There are two possibilities:

- (a) If $u \geq 0$, then $\hat{\mathbf{x}}$ is optimal for (MP). Terminate the column generation algorithm.
- (b) Otherwise, pick the column j which achieves the minimum, and add $\mathbf{A}_j$ to the subset of columns, i.e. $I \leftarrow I \cup \{j\}$. Go to Step 2.

Why does this procedure Step 3' work? Here is the argument, totally replying on LP duality theory.

1. If $u \geq 0$, then as shown under (6), $\hat{\mathbf{y}}$ is feasible for (DMP).
2. As we already know, $\hat{\mathbf{x}}$ is feasible for (MP). Therefore, we have found a primal solution $\hat{\mathbf{x}}$ and a dual solution $\hat{\mathbf{y}}$ feasible for the original (MP) and (DMP), respectively.
3. Furthermore, we know that $\mathbf{c}^T \hat{\mathbf{x}} = \mathbf{b}^T \hat{\mathbf{y}}$, because $\hat{\mathbf{x}}$ and $\hat{\mathbf{y}}$ are optimal for the restricted problems (RMP) and (DRMP), which are dual to each other. By the strong duality, their objective values must be equal.
4. Thus, by the corollary of the weak duality, $\hat{\mathbf{x}}$ and $\hat{\mathbf{y}}$ are optimal for (MP) and (DMP), respectively.